Module 15: Advanced Features

Cutting-Edge Blockchain Capabilities

Advanced Features Overview

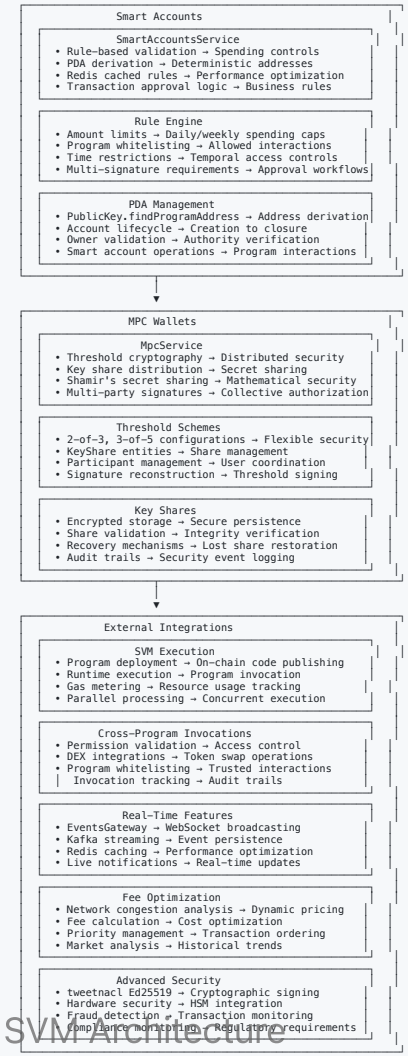
Next-Generation Capabilities

- **Smart Accounts**: Programmatic account control with validation rules
- **MPC Wallets**: Multi-party computation for enhanced security
- **SVM Execution**: Advanced program runtime with gas metering
- **Cross-Program Invocations**: Secure inter-program communication
- **Real-Time Features**: Live event streaming and notifications
- **Fee Optimization**: Dynamic pricing and cost optimization
- **Advanced Security**: Hardware security and fraud prevention

Innovation Focus

- **DeFi Integration**: Decentralized exchange operations
- **Institutional Features**: Enterprise-grade security and compliance

System Architecture



Smart Accounts Implementation

Rule-Based Validation Engine

```
@Injectable()
export class SmartAccountsService {
  constructor(
    private redis: Redis,
    private solanaConnection: Connection,
  ) {}

  async validateTransaction(
    smartAccountAddress: string,
    transaction: Transaction
  ): Promise<ValidationResult> {

    // Load account rules from cache
    const rules = await this.getAccountRules(smartAccountAddress);
    if (!rules) {
      return { valid: false, reason: 'Smart account not found' };
    }

    // Check account status
    if (rules.status !== 'ACTIVE') {
      return { valid: false, reason: 'Account is not active' };
    }

    // Validate spending limits
    const dailySpent = await this.getDailySpending(smartAccountAddress);
    const transactionAmount = this.calculateTransactionAmount(transaction);

    if (dailySpent + transactionAmount > rules.maxDailySpend) {
      return { valid: false, reason: 'Daily spending limit exceeded' };
    }

    // Check program whitelist
    const allowedPrograms = new Set(rules.allowedPrograms);
    for (const instruction of transaction.instructions) {
      if (!allowedPrograms.has(instruction.programId.toString())) {
        return { valid: false, reason: 'Program not whitelisted' };
      }
    }

    // Check time restrictions
    if (rules.timeRestrictions) {
      const now = new Date();
      if (!this.isWithinTimeWindow(now, rules.timeRestrictions)) {
        return { valid: false, reason: 'Transaction outside allowed time window' };
      }
    }

    // Multi-signature check
    if (rules.requiredSigners > 1) {
      const signatureCount = transaction.signatures.length;
      if (signatureCount < rules.requiredSigners) {
        return { valid: false, reason: 'Insufficient signatures' };
      }
    }

    return { valid: true };
  }

  private async getAccountRules(address: string): Promise<SmartAccountRules | null> {
    const cached = await this.redis.get(`smart-account:${address}:rules`);
    if (cached) {
      return JSON.parse(cached);
    }

    // Load from database if not cached
    const account = await this.smartAccountRepository.findOne({ where: { address } });
    if (account) {
      await this.redis.setex(`smart-account:${address}:rules`, 3600, JSON.stringify(account.rules));
```

MPC Wallet Implementation

Threshold Cryptography Service

```
@Injectable()
export class MpcService {
  constructor(
    private encryption: EncryptionService,
    private keySharesRepository: Repository<KeyShare>,
  ) {}

  async createMpcWallet(
    participants: string[],
    threshold: number,
    totalShares: number
  ): Promise<MpcWallet> {

    // Generate master key
    const masterKey = Keypair.generate();

    // Split key using Shamir's secret sharing
    const shares = this.splitKey(masterKey.secretKey, totalShares, threshold);

    // Encrypt and distribute shares
    const keyShares: KeyShare[] = [];
    for (let i = 0; i < participants.length; i++) {
      const encryptedShare = await this.encryption.encrypt(
        shares[i],
        participants[i]  // Encrypt with participant's public key
      );

      keyShares.push(this.keySharesRepository.create({
        walletId: masterKey.publicKey.toString(),
        participantId: participants[i],
        shareIndex: i,
        encryptedShare,
        status: 'ACTIVE'
      }));
    }

    await this.keySharesRepository.save(keyShares);

    return {
      address: masterKey.publicKey.toString(),
      threshold,
      totalShares,
      participants,
      status: 'CREATED'
    };
  }

  async signWithMpc(
    walletAddress: string,
    message: Uint8Array,
    participantSignatures: ParticipantSignature[]
  ): Promise<SignatureResult> {

    // Verify threshold met
    if (participantSignatures.length < this.threshold) {
      throw new BadRequestException('Insufficient signatures for threshold');
    }

    // Decrypt shares from signatures
    const decryptedShares: Uint8Array[] = [];
    for (const sig of participantSignatures) {
      const share = await this.keySharesRepository.findOne({
        where: {
          walletId: walletAddress,
          participantId: sig.participantId
        }
      });

      const decryptedShare = await this.encryption.decrypt(
        share.encryptedShare,
        sig.signature  // Use signature as decryption key
      );

      decryptedShares.push(decryptedShare);
    }

    // Reconstruct private key
    const reconstructedKey = this.reconstructKey(decryptedShares);

    // Sign message
    const signature = nacl.sign.detached(message, reconstructedKey);

    return {
      signature: Buffer.from(signature).toString('base64'),
      publicKey: walletAddress
    };
  }

  private splitKey(secretKey: Uint8Array, n: number, t: number): Uint8Array[] {
    // Implementation of Shamir's secret sharing
    // This is a simplified version - production would use a proper crypto library
    const shares: Uint8Array[] = [];

    for (let i = 0; i < n; i++) {
      // Generate polynomial coefficients
      const coefficients = [secretKey];
      for (let j = 1; j < t; j++) {
        coefficients.push(crypto.randomBytes(32));
      }

      // Evaluate polynomial at point i+1
      const share = this.evaluatePolynomial(coefficients, i + 1);
      shares.push(share);
    }
```

SVM Execution Engine

Gas Metering and Resource Management

```
@Injectable()
export class SvmService {
  constructor(private connection: Connection) {}

  async executeProgram(params: ProgramExecutionParams): Promise<ExecutionResult> {
    const startTime = Date.now();
    let computeUnitsUsed = 0;

    try {
      // Estimate compute units
      const estimatedCU = await this.estimateComputeUnits(params.instruction);

      // Check against limits
      if (estimatedCU > MAX_COMPUTE_UNITS) {
        throw new BadRequestException('Instruction exceeds compute unit limit');
      }

      // Set compute unit limit
      const setCULimitIx = ComputeBudgetProgram.setComputeUnitLimit({
        units: estimatedCU
      });

      // Build transaction with compute budget
      const transaction = new Transaction()
        .add(setCULimitIx)
        .add(params.instruction);

      // Add accounts
      transaction.recentBlockhash = (await this.connection.getRecentBlockhash()).blockhash;
      transaction.feePayer = params.feePayer;

      // Sign and send
      const signature = await this.connection.sendAndConfirmTransaction(transaction, [params.signer]);

      // Get actual compute units used (would need program logs parsing)
      computeUnitsUsed = await this.parseComputeUnitsFromLogs(signature);

      return {
        signature,
        success: true,
        computeUnitsUsed,
        executionTime: Date.now() - startTime
      };

    } catch (error) {
      return {
        signature: null,
        success: false,
        error: error.message,
        computeUnitsUsed,
        executionTime: Date.now() - startTime
      };
    }
  }

  private async estimateComputeUnits(instruction: TransactionInstruction): Promise<number> {
    const programId = instruction.programId.toString();

    // Base overhead
    let cu = 5000;

    // Program-specific estimates
    if (programId === SystemProgram.programId.toString()) {
      cu += this.estimateSystemProgramCU(instruction);
    } else if (programId === TOKEN_PROGRAM_ID.toString()) {
      cu += this.estimateTokenProgramCU(instruction);
    } else {
      // Conservative estimate for unknown programs
      cu += 10000;
    }
    return cu;
  }

  private estimateSystemProgramCU(instruction: TransactionInstruction): number {
```

DEX Integration and CPI

Decentralized Exchange Operations

```
@Injectable()
export class DexService {
  constructor(
    private cpiService: CpiService,
    private jupiterApi: JupiterApiService,
  ) {}

  async executeTokenSwap(swapParams: TokenSwapParams): Promise<SwapResult> {
    // Get quote from Jupiter
    const quote = await this.jupiterApi.getQuote({
      inputMint: swapParams.inputMint,
      outputMint: swapParams.outputMint,
      amount: swapParams.amount,
      slippageBps: swapParams.slippage * 100, // Convert to basis points
    });

    // Validate slippage
    if (quote.priceImpactPct > swapParams.maxPriceImpact) {
      throw new BadRequestException('Price impact too high');
    }

    // Create swap instruction via CPI
    const swapInstruction = await this.createJupiterSwapInstruction(quote);

    // Execute via CPI service
    const result = await this.cpiService.executeCpi({
      callerProgramId: this.dexProgramId,
      targetProgramId: quote.programId,
      instruction: swapInstruction,
      accounts: quote.accounts,
      requiresPermission: false, // DEX swaps are typically permissionless
    });

    return {
      signature: result.transactionId,
      inputAmount: swapParams.amount,
      outputAmount: quote.outputAmount,
      priceImpact: quote.priceImpactPct,
      fee: quote.fee
    };
  }

  async getLiquidityPools(): Promise<LiquidityPool[]> {
    // Query Jupiter for available pools
    const pools = await this.jupiterApi.getPools();

    return pools.map(pool => ({
      id: pool.id,
      tokenA: pool.tokenA,
      tokenB: pool.tokenB,
      liquidity: pool.liquidity,
      fee: pool.fee,
      volume24h: pool.volume24h
    }));
  }

  private async createJupiterSwapInstruction(quote: JupiterQuote): Promise<TransactionInstruction> {
    // This would integrate with Jupiter's instruction builder
    // Simplified for demonstration
    return new TransactionInstruction({
      programId: new PublicKey(quote.programId),
      keys: quote.accounts.map(acc => ({
        pubkey: new PublicKey(acc.pubkey),
        isSigner: acc.isSigner,
```

Real-Time Event Streaming

WebSocket Event Broadcasting

```
@Injectable()
export class RealTimeEventsService {
  constructor(
    private eventsGateway: EventsGateway,
    private kafkaProducer: KafkaProducerService,
    private redis: Redis,
  ) {}

  async broadcastTransactionEvent(event: TransactionEvent): Promise<void> {
    // Cache event for replay
    await this.redis.setex(
      `event:${event.signature}`,
      86400, // 24 hours
      JSON.stringify(event)
    );

    // Publish to Kafka for persistence
    await this.kafkaProducer.publishTransactionEvent(event);

    // Broadcast via WebSocket
    await this.eventsGateway.broadcastEvent({
      eventType: EventType.TRANSACTION_CONFIRMED,
      source: event.signature,
      data: event,
      slot: event.slot,
      signature: event.signature,
    });
  }

  async subscribeToAccount(accountAddress: string, clientId: string): Promise<void> {
    // Create subscription
    const subscription = await this.subscriptionService.createSubscription({
      clientId,
      eventTypes: [EventType.ACCOUNT_CHANGED],
      filters: {
        accountAddress,
      },
    });

    // Start monitoring account changes
    await this.connection.onAccountChange(
      new PublicKey(accountAddress),
      async (accountInfo) => {
        const event: AccountEvent = {
          address: accountAddress,
          data: accountInfo,
          slot: accountInfo.slot,
          timestamp: new Date(),
        };

        await this.broadcastAccountEvent(event);
      }
    );
  }

  async getEventHistory(
    eventType: EventType,
    startTime: Date,
    endTime: Date
  ): Promise<Event[]> {
    // Query from database with time range
    return await this.eventRepository.find({
      where: {
        eventType,
```

Fee Optimization Engine

Dynamic Fee Calculation

```
@Injectable()
export class FeeOptimizationService {
  constructor(
    private connection: Connection,
    private redis: Redis,
  ) {}

  async optimizeFee(
    transaction: Transaction,
    strategy: FeeOptimizationStrategy = 'BALANCED'
  ): Promise<OptimizedFee> {

    // Get network congestion
    const congestion = await this.analyzeNetworkCongestion();

    // Get historical fee data
    const historicalFees = await this.getHistoricalFeeData(24); // Last 24 hours

    // Calculate base fee
    const baseFee = await this.calculateBaseFee(transaction);

    // Apply optimization strategy
    let optimizedFee: number;

    switch (strategy) {
      case 'CONSERVATIVE':
        optimizedFee = this.applyConservativeStrategy(baseFee, congestion, historicalFees);
        break;
      case 'BALANCED':
        optimizedFee = this.applyBalancedStrategy(baseFee, congestion, historicalFees);
        break;
      case 'AGGRESSIVE':
        optimizedFee = this.applyAggressiveStrategy(baseFee, congestion, historicalFees);
        break;
      case 'PREDICTIVE':
        optimizedFee = await this.applyPredictiveStrategy(baseFee, historicalFees);
        break;
      default:
        optimizedFee = baseFee;
    }

    return {
      baseFee,
      optimizedFee,
      strategy,
      congestion,
      estimatedConfirmationTime: this.estimateConfirmationTime(optimizedFee, congestion),
      successProbability: this.calculateSuccessProbability(optimizedFee, historicalFees),
    };
  }

  private async analyzeNetworkCongestion(): Promise<'low' | 'medium' | 'high'> {
    const samples = await this.connection.getRecentPerformanceSamples(5);
    const avgTPS = samples.reduce((sum, s) => sum + s.numTransactions, 0) / samples.length;

    if (avgTPS > 5000) return 'high';
    if (avgTPS > 2000) return 'medium';
    return 'low';
  }

  private applyConservativeStrategy(
    baseFee: number,
    congestion: string,
    historicalFees: number[]
  ): number {
    const minHistoricalFee = Math.min(...historicalFees);
    return Math.max(baseFee, minHistoricalFee * 0.8);
  }

  private applyBalancedStrategy(
    baseFee: number,
    congestion: string,
    historicalFees: number[]
  ): number {
    const medianFee = this.calculateMedian(historicalFees);
    const congestionMultiplier = congestion === 'high' ? 2.0 :
                                 congestion === 'medium' ? 1.5 : 1.0;
    return medianFee * congestionMultiplier;
  }

  private applyAggressiveStrategy(
    baseFee: number,
    congestion: string,
    historicalFees: number[]
  ): number {
    const p95Fee = this.calculatePercentile(historicalFees, 95);
    return Math.max(baseFee * 3, p95Fee);
  }

  private async applyPredictiveStrategy(
    baseFee: number,
    historicalFees: number[]
```

Key Takeaways

Advanced Features Benefits

- **Smart Accounts**: Programmatic control with enterprise-grade security
- **MPC Wallets**: Distributed security for institutional use cases
- **SVM Execution**: High-performance parallel processing
- **DEX Integration**: Seamless token swap capabilities
- **Real-Time Events**: Live blockchain monitoring and notifications
- **Fee Optimization**: Cost-effective transaction processing

Production-Ready Capabilities

- **Scalable Architecture**: Multi-instance deployment with load balancing
- **Security First**: Comprehensive validation and fraud prevention
- **Performance Optimized**: Caching, parallel processing, and efficient algorithms